INTRODUCTION · EXPERIMENTAL DESIGN

Choose the right experimental unit

Before choosing a statistical test, identify what was independently treated or sampled. The number of measurements is not necessarily the number of independent experimental units.

Biological and technical replicates

A **biological replicate** is an independently prepared biological unit, such as a separate culture. It captures biological variability between units. A **technical replicate** is a repeated measurement of the same unit; it describes measurement variability, but does not create another independent culture.

Example · cell viability under one condition

Suppose four independently prepared cultures (B1–B4) are each measured three times. The experiment has **four biological units** and three technical measurements per unit—not twelve independent cultures.

Incorrect analysis	Appropriate handling
Count all 12 readings as independent samples (n = 12).	Average the three technical readings within each culture, then analyze one value per culture (n = 4).

Why pseudoreplication matters

Treating technical measurements as independent observations is pseudoreplication. It artificially inflates the apparent sample size, can underestimate uncertainty and standard errors, and can produce misleadingly small p-values. The analysis should reflect the number of independent biological units.

When observations are collected in matched conditions from the same biological units, preserve that unit's identifier so the measurements can be paired correctly. When each group's observations come from distinct units, use an independent-sample test.

In cstats: Set the biological-unit identifier with `cstats.set_reps()`. Use `cstats.aggregate()` to average technical measurements within factor-and-replicate groups. This retains one row per biological unit and condition for downstream comparisons.

Independent or paired?

Independence is a design property, not something a test can infer from the values. Choose the test according to how samples were collected and matched.

Comparison	Independent observations	Dependent / matched observations
Two groups · parametric	Welch's two-sample t-test	Paired t-test
Two groups · nonparametric	Mann–Whitney U / Wilcoxon rank-sum test	Wilcoxon signed-rank test
Multiple groups or factors	Conventional factorial ANOVA, when its assumptions and design are appropriate	Repeated-measures or mixed-effects methods, as appropriate for the design

How paired comparisons work in cstats

1. Register the biological-unit ID column with `cstats.set_reps()`.
2. Average technical replicates with `cstats.aggregate()`, if your design has them.
3. Use `paired = TRUE` with `cstats.t()`, `cstats.wilcox()`, `cstats.barplot.t()`, or `cstats.barplot.wilcox()`.

The paired functions match rows by the replicate ID column(s) registered with `cstats.set_reps()`, rather than relying on row order. Each ID must identify at most one observation in each compared group. Incomplete matches or missing outcome values are excluded with a warning; duplicate IDs within a group stop the comparison. If replicate IDs have not been set, a warning is raised and the paired operation stops.

```
data <- cstats.set_factors(data, c("Time", "Treatment"))
data <- cstats.set_vars(data, "Viability")
data <- cstats.set_reps(data, "BioRep")

data_agg <- cstats.aggregate(data)

# Match the same biological replicates across treatments at T48
paired_result <- cstats.t(
  data_agg, "Viability", base = "T48", paired = TRUE
)
paired_plot <- cstats.barplot.t(
  data_agg, "Viability", paired = TRUE
)
```

> **Design still matters:** Distribution, sample size, study design, and test assumptions also matter. The current cstats two-way ANOVA is a conventional factorial model; it does not itself account for repeated measures or random effects. Pairing is not a substitute for an appropriate repeated-measures or mixed-effects model when the design requires one.

Technical vs. Biological Replicates – Pseudoreplication

Example: cell viability under one experimental condition

Biological replicate

A culture independent of the others (B1, B2, B3, B4).

Captures variability between biological units.

Technical replicate

Three measurements of the same culture.

Captures measurement variability, not three new cultures.



4 biological units



3 measurements per unit

Technical vs. Biological Replicates – Pseudoreplication

Treating technical measurements as independent observations artificially inflates n

Incorrect: n = 12

4 cultures × 3 measurements = 12 rows

The 12 measurements are NOT 12 biological replicates.

Correct: n = 4

We average the 3 measurements from each culture.

We analyze 4 values, one per biological unit.

Pseudoreplication

- Artificially inflated sample size
- Underestimated uncertainty: standard errors too small
- Artificially low p-values

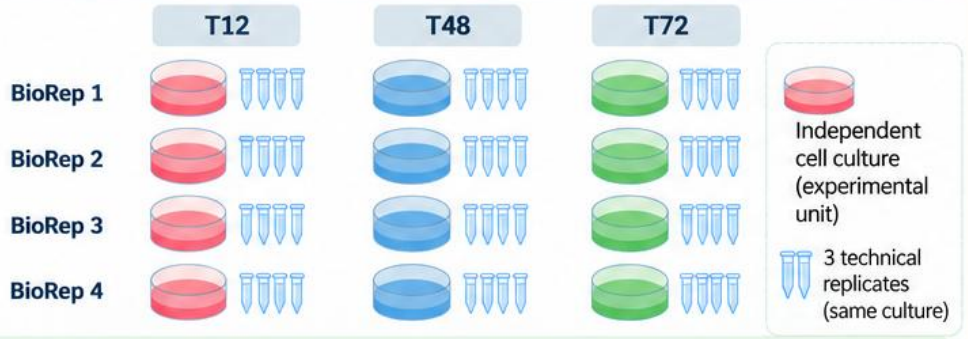
Biological Replicates – Independent vs. Dependent

Independent biological replicates

Each observation comes from a different experimental unit

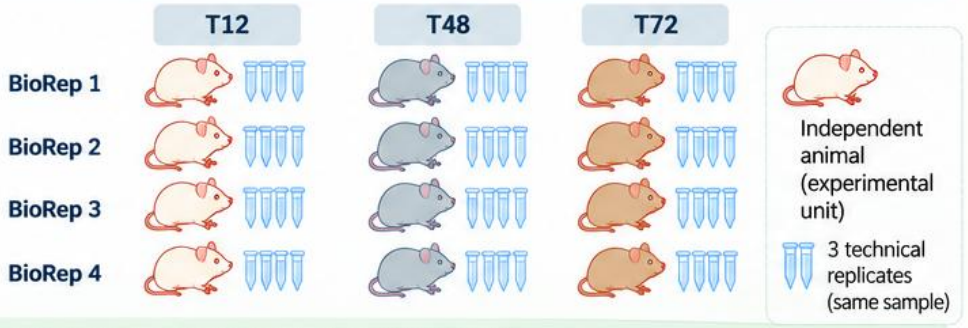
At each time point, we use different experimental units.
Observations are independent from each other.

Example 1: independent cell cultures at each time point



The cultures at T12, T48 and T72 are different.
All observations (labels B1–B4) correspond to independent units.

Example 2: different animals at each time point



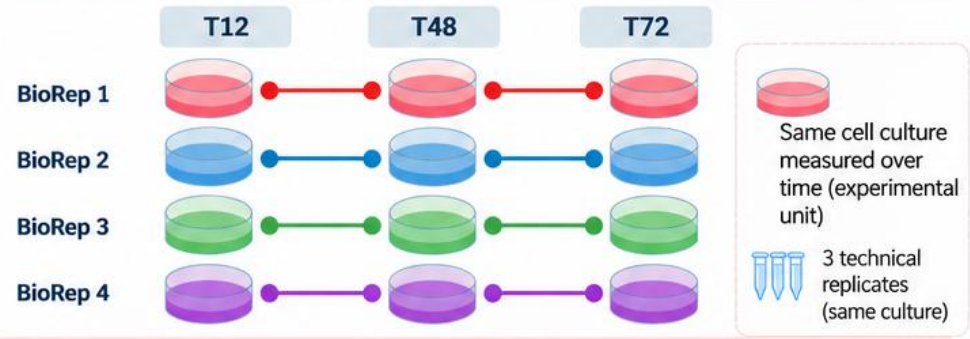
Different animals are used at each time point.
Observations are independent from each other.

Dependent biological replicates

Multiple observations come from the same experimental unit

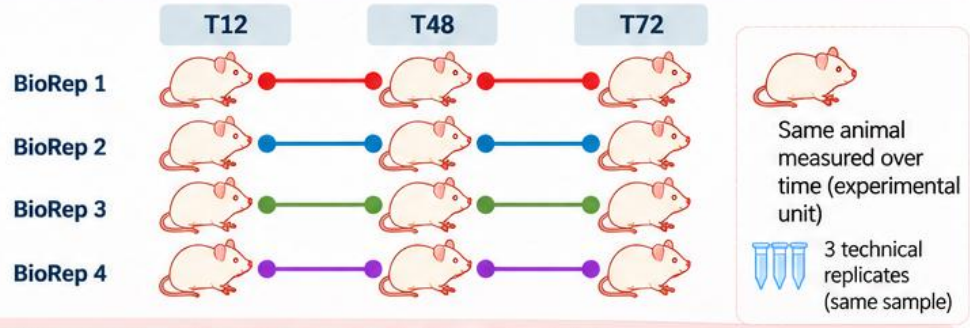
The same experimental units are measured repeatedly over time.
Observations from the same experimental unit are not independent (they are correlated).

Example 3: longitudinal follow-up of the same cell cultures



The same cell cultures are measured at T12, T48 and T72.
Repeated measures from the same culture are correlated.

Example 4: longitudinal follow-up of the same animals



The same animals are measured at T12, T48 and T72.
Repeated measures from the same animal are correlated.

Which Statistical Test Should We Use?

Independence is an assumption shared by many statistical tests

Comparison	Independent observations	Dependent observations
2 groups · parametric	Welch's t-test	Paired t-test
2 groups · nonparametric	Mann–Whitney U / Wilcoxon rank-sum	Wilcoxon signed-rank
Multiple groups / factors	Conventional factorial ANOVA	Mixed-effects model / repeated measures

Note: data distribution, test assumptions, and experimental design also matter.

FUNCTION REFERENCE

Understanding cstats.R

A practical guide to data setup, aggregation, statistical tests, and plots—including safe paired comparisons by replicate ID.

Based on the functions in `cstats.R`. The helper `cstats.parse` is intentionally excluded.

At a glance

The functions form a workflow: import a table, designate factors and outcomes, identify experimental replicates, optionally average technical replicates, then analyze or plot. These selections are stored as data-frame attributes: `F` (factors), `V` (outcomes), and `R` (replicate ID columns).

1. `cstats.read` — import a delimited file
2. `cstats.set_factors` — designate experimental factors
3. `cstats.set_vars` — designate measured outcomes
4. `cstats.set_reps` — designate replicate IDs
5. `cstats.aggregate` — average measurements within groups
6. `cstats.2wanova` — two-factor ANOVA for every selected outcome
7. `cstats.tukey` — Tukey-adjusted comparisons within groups
8. `cstats.wilcox` / `cstats.t` — pairwise tests
9. `cstats.barplot.tukey` / `cstats.barplot.wilcox` / `cstats.barplot.t` — plot and annotate results

> **Factor order matters:** For functions with two factors, the first factor defines groups/panels; levels of the second factor are compared within each group.

> **Paired comparisons:** With `paired = TRUE`, tests and plot annotations match samples using all replicate ID columns set by `cstats.set_reps()`. They do not rely on row order. Without those IDs, the function warns and stops rather than guessing.

cstats.read

```
cstats.read(path, separator = "\t")
```

Purpose. Reads a delimited text file as a data frame. The default tab separator is suitable for TSV files; pass another delimiter, such as `","`, for CSV data.

Arguments. `path` is the file path; `separator` is the field delimiter.

Returns. The imported data frame, after comma-decimal conversion. Any existing cstats metadata attributes (`F`, `V`, `R`) are removed.

```
data <- cstats.read("training/tutorial.txt")
data <- cstats.read("data.csv", separator = ",")
```

cstats.set_factors

```
cstats.set_factors(df, factors)
```

Purpose. Records the names of columns that define experimental groups or conditions in the `F` attribute.

Arguments. A data frame and a character vector of factor-column names.

Validation and return. Stops if a requested name is not a column; otherwise returns the data frame with `F` set.

```
data <- cstats.set_factors(data, c("Time", "Treatment"))
```

For two-factor analyses, the supplied order is important: the first factor is used for grouping and the second for within-group comparisons.

cstats.set_vars

```
cstats.set_vars(df, vars)
```

Purpose. Declares quantitative outcome columns for aggregation and analysis, storing their names in the `V` attribute.

Arguments. A data frame and a character vector of outcome-column names.

Validation and return. Stops if any requested column is missing; otherwise returns the data frame with `V` set.

```
data <- cstats.set_vars(data, c("Viability", "Marker"))
```

Aggregation takes means of these columns; `cstats.2wanova` fits one model per selected outcome. Single-outcome test and plot functions require `var` to be a selected variable.

cstats.set_reps

```
cstats.set_reps(df, replicates)
```

Purpose. Declares column(s) identifying replicate units and stores their names in the `R` attribute. These are sample identifiers, not replicate counts.

Arguments. A data frame and a character vector of replicate ID column names.

Validation and return. Stops if a requested name is missing; otherwise returns the data frame with `R` set. When several columns are provided, their combination forms the ID used to match a sample.

```
data <- cstats.set_reps(data, "BioRep")
# Multiple columns can jointly identify a sample:
data <- cstats.set_reps(data, c("Donor", "BioRep"))
```

> For paired tests, IDs must identify at most one observation in each compared group. Aggregate technical replicates first when necessary so each ID has one measurement per condition.

cstats.aggregate

```
cstats.aggregate(df)
```

Purpose. Averages each selected outcome over rows sharing the same factor and replicate identifiers, for example combining technical replicates while preserving biological replicate units.

Requirements and behavior. Requires factors (`F`) and outcomes (`V`). If `R` is unset, prints a warning and groups only by factors, averaging all rows in each factor combination.

Returns. A data frame with one row per group and mean outcome values. The `F`, `V`, and `R` attributes are retained.

```
data_agg <- cstats.aggregate(data)
```

> Without replicate IDs, biological replicates are also averaged together. Set `R` correctly if independent units must remain separate for downstream analysis.

cstats.2wanova

```
cstats.2wanova(df)
```

Purpose. Fits a two-way linear model with interaction for every outcome in `V`, using the two factors in `F`. Each model is `outcome ~ factor1 * factor2`, including main effects and their interaction.

Requirements. Factor and outcome attributes must be set, and exactly two factors must be selected.

Returns. A named list of base R ANOVA tables, one per outcome.

```
anova_results <- cstats.2wanova(data_agg)
anova_results$Viability
```

> Uses base R's sequential ANOVA table. Interpretation depends on design, model assumptions, and factor order. Assumptions are not checked and p-values are not adjusted across outcomes.

cstats.tukey

```
cstats.tukey(df, var)
```

Purpose. Compares levels of factor 2 within each level of factor 1, using estimated marginal means from a two-factor ANOVA with interaction.

Requirements. Exactly two factors and at least one selected outcome must be set. `var` names the outcome to analyze.

Adjustment and return. Pairwise contrasts use Tukey adjustment. Returns the `emmeans` pairwise-comparison result, including contrasts and inferential statistics.

```
tukey_results <- cstats.tukey(data_agg, "Viability")
summary(tukey_results)
```

The implementation uses `var` directly and does not explicitly check that it appears in the `V` attribute.

cstats.wilcox

```
cstats.wilcox(df, var, base = NULL, altrn = "two.sided", paired = FALSE)
```

Purpose. Runs every unordered pairwise Wilcoxon comparison for an outcome. With one factor, it compares all pairs of its levels. With two factors, it compares factor-2 levels within the specified level of factor 1 (`base`).

Arguments. `var` must be in `V`. With two factors, `base` must be a level of factor 1. `altrn` selects `"two.sided"`, `"less"`, or `"greater"`. `paired` defaults to `FALSE`.

Test and return. By default, performs the independent-sample Wilcoxon rank-sum test. With `paired = TRUE`, matches observations by the ID column(s) set with `cstats.set_reps()` and performs the paired Wilcoxon signed-rank test. Returns a named list. No multiple-comparison adjustment is applied.

```
wilcox_results <- cstats.wilcox(data_agg, "Viability", base = "T48")
paired_results <- cstats.wilcox(
  data_agg, "Viability", base = "T48", paired = TRUE
)
```

cstats.t

```
cstats.t(df, var, base = NULL, altrn = "two.sided", paired = FALSE)
```

Purpose. Runs pairwise t-tests using the same comparison scheme as `cstats.wilcox`.

Arguments. `var` must be in `V`. With two factors, `base` must be a level of factor 1. `altrn` selects the alternative hypothesis. `paired` defaults to `FALSE`.

Test and return. By default, performs Welch's two-sample t-test. With `paired = TRUE`, matches observations by the ID column(s) set with `cstats.set_reps()` and performs a paired t-test. Returns a named list. No multiple-comparison adjustment is applied.

```
t_results <- cstats.t(data_agg, "Viability", base = "T48")
paired_results <- cstats.t(
  data_agg, "Viability", base = "T48", paired = TRUE
)
```

cstats.barplot.tukey

```
cstats.barplot.tukey(df, var)
```

Purpose. Creates one plot for each level of factor 1. Each plot shows factor-2 group means, mean ± standard deviation error bars, and jittered individual observations.

Annotations. Fits a two-factor interaction model and calculates Tukey-adjusted factor-2 comparisons within each factor-1 level. Significant contrasts are marked with brackets and stars.

Requirements and return. Exactly two factors must be set; `var` must appear in `V`. Returns a named list of ggplot objects, one per factor-1 level.

```
plots <- cstats.barplot.tukey(data_agg, "Viability")
plots[[1]]
```

Star cutoffs: p < 0.05 (*), p < 0.01 (**), and p < 0.001 (***). Nonsignificant contrasts are not annotated.

cstats.barplot.wilcox

```
cstats.barplot.wilcox(df, var, paired = FALSE)
```

Purpose. Produces one plot per factor-1 level, showing factor-2 means, mean ± SD error bars, and jittered observations.

Annotations. Tests every pair of factor-2 levels within each factor-1 level. By default uses the Wilcoxon rank-sum test; with `paired = TRUE`, matches by the replicate ID column(s) and uses the paired signed-rank test. Plot tests are two-sided.

Requirements and return. Exactly two factors must be set; `var` must be in `V`. Returns a named list of ggplot objects.

```
plots <- cstats.barplot.wilcox(data_agg, "Viability")
paired_plots <- cstats.barplot.wilcox(
  data_agg, "Viability", paired = TRUE
)
```

> Paired annotations require IDs from `cstats.set_reps()`. IDs missing from either condition and rows with missing outcomes are excluded with warnings; duplicate IDs within a condition stop the comparison. No multiple-comparison adjustment is applied.

cstats.barplot.t

```
cstats.barplot.t(df, var, paired = FALSE)
```

Purpose. Produces one plot per factor-1 level, showing factor-2 means, mean ± SD error bars, and jittered individual observations.

Annotations. Tests every pair of factor-2 levels within each factor-1 level. By default uses Welch's two-sample t-test; with `paired = TRUE`, matches by the replicate ID column(s) and uses a paired t-test. Plot tests are two-sided.

Requirements and return. Exactly two factors must be set; `var` must be in `V`. Returns a named list of ggplot objects.

```
plots <- cstats.barplot.t(data_agg, "Viability")
paired_plots <- cstats.barplot.t(
  data_agg, "Viability", paired = TRUE
)
```

> Paired annotations require IDs from `cstats.set_reps()`. IDs missing from either condition and rows with missing outcomes are excluded with warnings; duplicate IDs within a condition stop the comparison. No multiple-comparison adjustment is applied.

Putting the workflow together

```
data <- cstats.read("training/tutorial.txt")
data <- cstats.set_factors(data, c("Time", "Treatment"))
data <- cstats.set_vars(data, "Viability")
data <- cstats.set_reps(data, "BioRep")

data_agg <- cstats.aggregate(data)

anova_results <- cstats.2wanova(data_agg)
tukey_results <- cstats.tukey(data_agg, "Viability")
t_results <- cstats.t(
  data_agg, "Viability", base = "T48", paired = TRUE
)
plots <- cstats.barplot.t(
  data_agg, "Viability", paired = TRUE
)
```

Paired-comparison checks

- Call `cstats.set_reps()` before paired tests or plots; the returned data frame must be used.
- Every replicate ID (or combination of IDs) should identify at most one measurement in each condition. Aggregate technical replicates first.
- Rows are joined by replicate ID, so their order in the data frame does not matter.
- Incomplete pairs and rows with missing outcomes are excluded with warnings; duplicated IDs within a comparison group stop the paired comparison.
- Pairwise tests do not adjust p-values for multiple comparisons. Plot error bars show standard deviations, not standard errors or confidence intervals.

Function behavior described here reflects the current implementation in `cstats.R`. This guide is not a substitute for statistical advice.